

Studentu pazymiu programa
v2.0

Sugeneruota Doxygen 1.17.0

skyrius 1

Studentų pažymių programa - v2.0

C++17 programa studentų pažymiams skaičiuoti, rūšiuoti ir grupuoti. Naudojamos OOP konstrukcijos: paveldėjimas, polimorfizmas, Rule of Five.

1.1 Įdiegimas

1.1.1 Reikalavimai

- C++17 kompiliatorius (g++ 9+, clang++ 9+, MSVC 2019+)
- CMake 3.14+ (*rekomenduojama*) arba make
- (*neprivaloma*) Doxygen + LaTeX dokumentacijai generuoti
- (*neprivaloma*) `catch.hpp` unit testams

1.1.2 Kompiliavimas su CMake

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

1.1.3 Kompiliavimas su Makefile (Unix)

```
make          # sukompiluoja viską
make Uzd2     # tik pagrindinę programą
make testas   # tik paprastus testus
make unit_testai # Catch2 testus (reikia catch.hpp)
make docs     # Doxygen dokumentaciją
make clean    # ištrina sukompiliuotus failus
```

1.1.4 Kompiliavimas rankiniu būdu

```
g++ -O2 -std=c++17 Uzd2.cpp vektorius.cpp testavimas.cpp funkcijos.cpp -o Uzd2
```

1.2 Naudojimas

1.2.1 Pagrindinė programa

```
./Uzd2
```

Interaktyvus meniu - leidžia įvesti studentus ranka, generuoti, skaityti iš failo, rūšiuoti ir skaidyti į grupes.

1.2.2 Tyrimo programos

```
./stud_Vector # sugeneruoja Data/ ir matuoja vector greitį
./stud_List   # naudoja tuos pačius Data/ failus
./stud_Deque  # naudoja tuos pačius Data/ failus
```

1.2.3 Testai

```
./testas          # paprastas rankinis testas (v1.2/v1.5)
./unit_testai     # pilnas Catch2 unit test rinkinys (v2.0)
```

1.2.4 Dokumentacijos generavimas

doxygen Doxyfile

```
# HTML - atidaromas naršyklėje
open docs/html/index.html      # macOS
xdg-open docs/html/index.html  # Linux
start docs/html/index.html     # Windows

# PDF - kompiliuojamas iš LaTeX
cd docs/latex
make                          # arba: pdflatex refman.tex
```

Jei nėra pdflatex lokaliai - įkelkite docs/latex/* į [Overleaf](#) ir sukompiliuokite ten.

1.3 Failų struktūra

```
.
+-- struktura.h          - klasės (Zmogus, Studentas), Doxygen komentarai
+-- funkcijos.cpp        - pagalbinės funkcijos
+-- vektorius.cpp        - meniu logika
+-- testavimas.cpp       - greičio tyrimas
+-- Uzd2.cpp             - pagrindinis įėjimas
+-- stud_Vector.cpp      - konteinerio tyrimas (vector)
+-- stud_List.cpp        - konteinerio tyrimas (list)
+-- stud_Deque.cpp       - konteinerio tyrimas (deque)
+-- testas.cpp           - paprastas rankinis testas
+-- unit_testai.cpp      - Catch2 unit testai
+-- CMakeLists.txt       - build konfigūracija
+-- Makefile             - alternatyvus build (Unix)
+-- Doxyfile             - Doxygen konfigūracija
+-- .gitignore
+-- README.md
```

1.4 Releasai

1.4.1 v0.1 - pradinė versija

Struct [Studentas](#), rankinis įvedimas, vidurkio/medianos skaičiavimas.

1.4.2 v0.2

Skaitymas iš failo, išvedimas į failą.

1.4.3 v0.3

Automatinis duomenų generavimas, klaidų apdorojimas.

1.4.4 v0.4

Skaidymas į kietus/tinginius, failo kūrimo ir apdorojimo tyrimai.

1.4.5 v1.0

Trys atskiros tyrimo programos ([vector](#), [list](#), [deque](#)) su dviem skaidymo strategijomis (S1, S3).

1.4.6 v1.1

Perėjimas nuo `struct` prie `class`. Privatūs laukai, getter'iai, setter'iai.

1.4.7 v1.2 - Rule of Five + I/O operatoriai

- Realizuota pilna penkių metodų taisyklė
- Perdengti `operator<<` ir `operator>>`
- Pridėtas testavimo failas `testas.cpp`

1.4.8 v1.5 - Paveldėjimas

- Sukurta abstrakti bazinė klasė `Zmogus`
- `Studentas` paveldi iš `Zmogus`
- Demonstruojamas virtualus dispatch ir polimorfizmas

1.4.9 v2.0 - Dokumentacija + Unit testai

- Doxygen dokumentacija (HTML + PDF)
- Catch2 unit testai
- Švari repozitorija (`.gitignore`)

1.5 v1.2 - Rule of Five paaiškinimas

Kai klasė valdo resursus (vector, string), reikia aprašyti penkis metodus:

#	Metodas	Funkcija
1	<code>~Studentas()</code>	Atlaisvina atmintį
2	<code>Studentas(const Studentas&)</code>	Gili kopija
3	<code>operator=(const Studentas&)</code>	Kopijavimo priskyrimas
4	<code>Studentas(Studentas&&) noexcept</code>	Move - be kopijavimo
5	<code>operator=(Studentas&&) noexcept</code>	Move priskyrimas

1.5.1 I/O operatoriai

Operatorius	Paskirtis	Pavyzdys
<code>operator<<</code>	Išvedimas į ekraną/failą	<code>cout << st; failas << st;</code>
<code>operator>></code>	Skaitymas iš klaviatūros/failo	<code>cin >> st; failas >> st;</code>

Formatas `operator>>`: `Vardas Pavardė ND1 ND2 ... NDn Egzaminas (paskutinis skaičius = egzaminas)`

1.5.2 Duomenų įvedimo būdai

Būdas	Kaip	Vidinis mechanizmas
Rankinis	Meniu -> 1	<code>cin >>, gautiSkaiciu()</code>
Automatinis	Meniu -> 3	<code>genVarda(), genPazymius()</code>
Iš failo	Meniu -> 4	<code>operator>></code>

1.5.3 Duomenų išvedimo būdai

Būdas	Kaip	Vidinis mechanizmas
Į ekraną	Po rūšiavimo	cout << st per <code>spausdintiRezultatus()</code>
Į failą	Po rūšiavimo	ofstream << st per <code>spausdintiRezultatus()</code>

1.6 v1.5 - Klasių hierarchija

```
Zmogus (abstrakti)
|
+-- Studentas
```

1.6.1 Kodėl Zmogus abstrakti

Klasėje yra gryna virtuali funkcija:

```
virtual void spausdinti(ostream& os) const = 0;
```

= 0 neleidžia sukurti `Zmogus` `z`; - kompiliatorius išmes klaidą.

1.6.2 Kritinis virtualus destruktorius

```
virtual ~Zmogus() = default; // BŪTINA paveldėjime
```

Be `virtual`, ištrinant per bazinę rodyklę (`delete zmogus_ptr`), iškvietumėme tik bazinį destruktorių - `Studento` `vector` ir kt. liktų neišvalyti.

1.6.3 Polimorfizmas

```
vector<unique_ptr<Zmogus> zmones;
zmones.push_back(make_unique<Studentas> (...));

for (const auto& z : zmones)
    cout << *z; // virtual dispatch -> Studentas::spausdinti
```

1.7 v2.0 - Unit testai

Naudojama `Catch2` - populiarus C++ testavimo framework'as (single-header).

1.7.1 Setup

Atsisiųskite `catch.hpp` į projekto katalogą:
`wget https://github.com/catchorg/Catch2/releases/download/v2.13.10/catch.hpp`

1.7.2 Sintaksė

```
TEST_CASE("aprasymas", "[tag1][tag2]") {
    Studentas st;
    REQUIRE(st.egz() == 0); // testas FAIL jei nepavyksta
    CHECK(st.vardas() == ""); // tęsia net jei nepavyksta

    SECTION("atskira testo dalis") {
        // ...
    }
}
```

1.7.3 Padengiamos sritys

TEST_CASE	Tag'as	Aprašas
Numatytasis konstruktorius	↔ [konstruktoriai]↔	Tikrina tuščio objekto reikšmės
Pilnas konstruktorius	↔ [konstruktoriai]↔	Skaičiavimas iš pradinių duomenų
Copy constructor	[rule_of_five]↔ [copy]	Gili kopija
Copy assignment	[rule_of_five]↔ [copy]	Savipriskyrimas
Move constructor	[rule_of_five]↔ [move]	Originalo ištuštinimas
Move assignment	[rule_of_five]↔ [move]	Saviperkėlimas
Destruktorius	[rule_of_↔ five]	Automatinis atminties valymas
Zmogus abstraktumas	↔ [abstrakti]↔	static_assert patvirtina
Virtualus dispatch	[abstrakti]↔ [polimorfizmas]	Per Zmogus& nuorodą
operator<<	↔ [io]↔	Visi laukai išvedami
operator>>	↔ [io]↔	Formato analizė
Skaičiavimai	↔ [skaiciavimas]↔	Vidurkis, mediana
Kraštiniai atvejai	[krastiniai_↔ atvejai]	Tuščias studentas

1.7.4 Paleidimas

```
./unit_testai           # visi testai
./unit_testai -s        # rodo ir praėjusius
./unit_testai "[copy]"  # tik tam tikras tag'ų grupes
```

1.8 Konteinerių tyrimo rezultatai

Testavimo sistema: (ASUS VIVOBOOK S 14)

Parametras	Reikšmė
CPU	AMD Ryzen AI 9
RAM	24 GB
Saugykla	1 TB SSD (NVMe)
OS	Windows 11

Parametras	Reikšmė
Kompiliatorius	g++

Žemiau pateikti rezultatai, gauti testavimo metu (su release):

1.8.1 Nuskaitymas (s)

Failas	vector	list	deque
1 000 įrašų	0.0050	0.0042	0.0050
10 000 įrašų	0.0411	0.0407	0.0406
100 000 įrašų	0.3905	0.3891	0.3811
1 000 000 įrašų	3.7790	3.7911	3.7534
10 000 000 įrašų	38.2739	50.6590	49.0184

Nuskaitymas visur panašus — skirtumai mažesni nei failo I/O triukšmas.

1.8.2 Rūšiavimas (s)

Failas	vector	list	deque
1 000 įrašų	0.0001	0.0000	0.0001
10 000 įrašų	0.0009	0.0007	0.0011
100 000 įrašų	0.0073	0.0193	0.0128
1 000 000 įrašų	0.0855	0.5396	0.2673
10 000 000 įrašų	0.7860	12.2730	4.2492

`std::vector` rūšiuojamas greičiausiai dėl gretimos atminties (CPU talpykla). `list` yra $\sim 2.5\times$ lėtesnis 100k atveju. `deque` artimas vektoriui.

1.9 Skaidymo strategijos

1.9.1 S1 — du nauji konteineriai (`copy_if`)

1.9.2 Skaidymas (s)

Failas	vector	list	deque
1 000 įrašų	0.0002	0.0001	0.0001
10 000 įrašų	0.0016	0.0021	0.0014
100 000 įrašų	0.0215	0.0330	0.0236
1 000 000 įrašų	0.2446	0.3862	0.2574
10 000 000 įrašų	3.1586	5.2494	3.2320

Originalas nekeičiamas. Kiekvienas studentas saugomas **dviejose** vietose atmintyje. Du pilni praėjimai per duomenis.

1.9.3 S3 — `stable_partition + move/splice`

1.9.4 Skaidymas (s)

Failas	vector	list	deque
1 000 įrašų	0.0000	0.0000	0.0000
10 000 įrašų	0.0001	0.0001	0.0003
100 000 įrašų	0.0011	0.0056	0.0045
1 000 000 įrašų	0.0130	0.0736	0.0636
10 000 000 įrašų	0.1820	0.8060	0.7349

Duomenys tvarkomi **vietoje** — nereikia papildomos atminties iteracijoms. Vargšiškai perkeliama (`move`) be kopijavimo. `list` versijoje vietoj `move` naudojamas `splice()` — tik rodyklių pakeitimas.

1.9.5 v1.1

Struct pakeista į Class Su visual studio

Skaidymas naudojant vector (be optimizacijos):

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0001
100 000 įrašų	0.0011
1 000 000 įrašų	0.0125
10 000 000 įrašų	0.2181

(su /O1)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0001
100 000 įrašų	0.0010
1 000 000 įrašų	0.0124
10 000 000 įrašų	0.3042

(su /O2)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0001
100 000 įrašų	0.0009
1 000 000 įrašų	0.0133
10 000 000 įrašų	0.1656

Su cmake:

(su /O1)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0001
100 000 įrašų	0.0021
1 000 000 įrašų	0.0273
10 000 000 įrašų	0.3471

(su /O2)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0003
100 000 įrašų	0.0025
1 000 000 įrašų	0.0282
10 000 000 įrašų	0.3588

(su /O3)

Failas	vector
1 000 įrašų	0.0000
10 000 įrašų	0.0003
100 000 įrašų	0.0023
1 000 000 įrašų	0.0272
10 000 000 įrašų	0.4811

Nors naudojant cmake dalinimas užtrunka ilgiau (apie 2x), tačiau failų nuskaitymas vyks ta kelis kart greičiau. Tokio dydžio duomenimis programa bendrai veikia greičiau. O naudojant Visual Studio ar CMake vis tiek bus greičiau nei naudojant struct be optimizacijos.

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Zmogus	??
Studentas	??

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

[Studentas](#)

Paveldėjusi klasė iš [Zmogus](#) — atstovauja studentui su pažymiais ir egzaminu ??

[Zmogus](#)

Abstrakti bazinė klasė, aprašanti bendrus žmogaus atributus ??

skyrius 4

Failo Indeksas

4.1 Failai

Visų failų sąrašas su trumpais aprašymais:

funkcijos.cpp	??
struktura.h	
Klasų apibrėžimai: Zmogus (abstrakti bazė) ir Studentas (paveldėjusi)	??
Uzd2.cpp	??
vektorius.cpp	??

skyrius 5

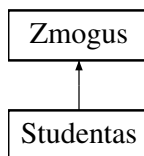
Klasės Dokumentacija

5.1 Studentas Klasė

Paveldėjusi klasė iš [Zmogus](#) — atstovauja studentui su pažymiais ir egzaminu.

```
#include <struktura.h>
```

Paveldimumo diagrama Studentas:



Vieši Metodai

- [Studentas](#) ()=default
Numatytasis konstruktorius — sukuria tuščią studentą.
- [Studentas](#) (string v, string p, vector< int > paz, int egz, int metodas)
Pilnas konstruktorius su iš karto atliekamu skaičiavimu.
- void [spausdinti](#) (ostream &os) const override
Perdengia bazinę [spausdinti\(\)](#) funkciją.
- const vector< int > & [paz](#) () const
Grąžina namų darbų pažymių vektorių (const nuoroda).
- int [egz](#) () const
Grąžina egzamino balą.
- double [galVid](#) () const
Grąžina galutinį pažymį (vidurkio metodu).
- double [galMed](#) () const
Grąžina galutinį pažymį (medianos metodu).
- void [setEgz](#) (int e)
Nustato egzamino balą.
- void [addPazymys](#) (int p)
Prideda vieną pažymį prie ND sąrašo.
- void [nustatytiEgzlsGalo](#) ()
Paskutinį pažymį iš sąrašo perkelia į egzamino lauką.
- double [skaiciuotiVidurki](#) () const
Apskaiciuoja namų darbų vidurkį.
- double [skaiciuotiMediana](#) () const
Apskaiciuoja namų darbų medianą.
- void [apskaiciuoti](#) (int metodas)

Apskaičiuoja galutinį pažymį pagal pasirinktą metodą.

Rule of Five

- `~Studentas ()` override=default
Destruktorius.
- `Studentas (const Studentas &kitas)`
Kopijavimo konstruktorius.
- `Studentas & operator= (const Studentas &kitas)`
Kopijavimo priskyrimo operatorius.
- `Studentas (Studentas &&kitas) noexcept`
Perkėlimo konstruktorius (move).
- `Studentas & operator= (Studentas &&kitas) noexcept`
Perkėlimo priskyrimo operatorius.

Vieši Metodai inherited from Zmogus

- `Zmogus ()`=default
Numatytasis konstruktorius — sukuria tuščią objektą.
- `Zmogus (string v, string p)`
Pilnas konstruktorius.
- `const string & vardas () const`
Grąžina vardą.
- `const string & pavarde () const`
Grąžina pavardę.
- `void setVardas (const string &v)`
Nustato vardą.
- `void setPavarde (const string &p)`
Nustato pavardę.
- `Zmogus (const Zmogus &)=default`
- `Zmogus (Zmogus &&) noexcept=default`
- `Zmogus & operator= (const Zmogus &)=default`
- `Zmogus & operator= (Zmogus &&) noexcept=default`
- `virtual ~Zmogus ()=default`
Virtualus destruktoriaus.

Privatūs Atributai

- `vector< int > paz_`
Namų darbų pažymiai.
- `int egz_ = 0`
Egzamino balas.
- `double gal_vid_ = 0.0`
Galutinis pažymys (vidurkio metodu).
- `double gal_med_ = 0.0`
Galutinis pažymys (medianos metodu).

Draugai

- `istream & operator>> (istream &is, Studentas &st)`
Ivesties operatorius Studento objektui.

Additional Inherited Members

Apsaugoti Atributai inherited from **Zmogus**

- string **vardas_**
Žmogaus vardas.
- string **pavarde_**
Žmogaus pavardė

5.1.1 Smulkus aprašymas

Paveldėjusi klasė iš **Zmogus** — atstovauja studentui su pažymiais ir egzaminu.

Paveldi vardą ir pavardę iš **Zmogus**, prideda namų darbų pažymius (*paz_*), egzamino balą (*egz_*) ir du galutinio pažymio variantus (vidurkis ir mediana).

5.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

5.1.2.1 Studentas() [1/4]

```
Studentas::Studentas () [default]
```

Numatytasis konstruktorius — sukuria tuščią studentą.

5.1.2.2 Studentas() [2/4]

```
Studentas::Studentas (
    string v,
    string p,
    vector< int > paz,
    int egz,
    int metodas) [inline]
```

Pilnas konstruktorius su iš karto atliekamu skaičiavimu.

Parametrai

<i>v</i>	Vardas
<i>p</i>	Pavardė
<i>paz</i>	Namų darbų pažymių vektorius
<i>egz</i>	Egzamino balas
<i>metodas</i>	Skaičiavimo metodas: 1 — vidurkis, 2 — mediana, 3 — abu

5.1.2.3 ~Studentas()

```
Studentas::~Studentas () [override], [default]
```

Destruktorius.

5.1.2.4 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & kitas) [inline]
```

Kopijavimo konstruktorius.

Parametrai

<i>kitas</i>	Originalas, iš kurio kopijuojama
--------------	----------------------------------

Sukuria gilią kopiją — vector ir string laukai nukopijuojami atskirai.

5.1.2.5 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && kitas) [inline], [noexcept]
```

Perkėlimo konstruktorius (move).

Parametrai

<i>kitas</i>	Objektas, iš kurio perduodami resursai
--------------	--

`noexcept` — leidžia `std::vector` naudoti šį konstruktorių vietoj kopijavimo realokavimo metu.

5.1.3 Metodų Dokumentacija

5.1.3.1 addPazymys()

```
void Studentas::addPazymys (
    int p) [inline]
```

Prideda vieną pažymį prie ND sąrašo.

5.1.3.2 apskaiciuoti()

```
void Studentas::apskaiciuoti (
    int metodas) [inline]
```

Apskaičiuoja galutinį pažymį pagal pasirinktą metodą.

Parametrai

<i>metodas</i>	1 — vidurkis, 2 — mediana, 3 — abu
----------------	------------------------------------

Formulė: $\text{galutinis} = 0.4 \times \text{NDvidurkis} + 0.6 \times \text{egzaminas}$

5.1.3.3 egz()

```
int Studentas::egz () const [inline]
```

Grąžina egzamino balą.

5.1.3.4 galMed()

```
double Studentas::galMed () const [inline]
```

Grąžina galutinį pažymį (medianos metodu).

5.1.3.5 galVid()

```
double Studentas::galVid () const [inline]
```

Grąžina galutinį pažymį (vidurkio metodu).

5.1.3.6 nustatytiEgzIsGalo()

```
void Studentas::nustatytiEgzIsGalo () [inline]
```

Paskutinį pažymį iš sąrašo perkelia į egzamino lauką.
Naudojama skaitant iš failo, kur paskutinis skaičius eilutėje laikomas egzamino balu.

5.1.3.7 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & kitas) [inline]
```

Kopijavimo priskyrimo operatorius.

Parametrai

<i>kitas</i>	Originalas
--------------	------------

Gražina

Nuoroda į šį objektą

Apsaugotas nuo savipriskyrimo ($a = a$).

5.1.3.8 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && kitas) [inline], [noexcept]
```

Perkėlimo priskyrimo operatorius.

5.1.3.9 paz()

```
const vector< int > & Studentas::paz () const [inline]
```

Gražina namų darbų pažymių vektorių (const nuoroda).

5.1.3.10 setEgz()

```
void Studentas::setEgz (
    int e) [inline]
```

Nustato egzamino balą.

5.1.3.11 skaiciuotiMediana()

```
double Studentas::skaiciuotiMediana () const [inline]
```

Apskaičiuoja namų darbų medianą.

Gražina

Mediana arba 0.0 jei pažymių nėra.

5.1.3.12 skaiciuotiVidurki()

```
double Studentas::skaiciuotiVidurki () const [inline]
```

Apskaičiuoja namų darbų vidurkį.

Gražina

Aritmetinis vidurkis arba 0.0 jei pažymių nėra.

5.1.3.13 spausdinti()

```
void Studentas::spausdinti (
    ostream & os) const [inline], [override], [virtual]
```

Perdengia bazinę [spausdinti\(\)](#) funkciją.

Parametrai

<i>os</i>	Išvesties srautas
-----------	-------------------

Formatuoja studento duomenis į vieną eilutę su ND, egzaminu ir galutiniais pažymiais (jei jie apskaičiuoti). Realizuoja [Zmogus](#).

5.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

5.1.4.1 operator>>

```
istream & operator>> (
    istream & is,
    Studentas & st) [friend]
```

Ivesties operatorius Studento objektui.

Parametrai

<i>is</i>	Ivesties srautas
<i>st</i>	Studentas , į kurį skaitoma

Gražina

Nuoroda į srautą

Tikisi vienos eilutės: Vardas Pavardė ND1 ND2 ... NDn Egzaminas Paskutinis skaičius laikomas egzamino balu.

5.1.5 Atributų Dokumentacija

5.1.5.1 egz_

```
int Studentas::egz_ = 0 [private]
```

Egzamino balas.

5.1.5.2 gal_med_

```
double Studentas::gal_med_ = 0.0 [private]
```

Galutinis pažymys (medianos metodu).

5.1.5.3 gal_vid_

```
double Studentas::gal_vid_ = 0.0 [private]
```

Galutinis pažymys (vidurkio metodu).

5.1.5.4 paz_

```
vector<int> Studentas::paz_ [private]
```

Namų darbų pažymiai.

Dokumentacija šiai klasei sugeneruota iš šio failo:

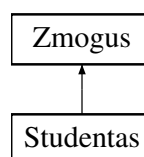
- [struktura.h](#)

5.2 Zmogus Klasė

Abstrakti bazinė klasė, aprašanti bendrus žmogaus atributus.

```
#include <struktura.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- `Zmogus ()`=default
Numatytasis konstruktorius — sukuria tuščią objektą.
- `Zmogus (string v, string p)`
Pilnas konstruktorius.
- `const string & vardas () const`
Grąžina vardą.
- `const string & pavarde () const`
Grąžina pavardę.
- `void setVardas (const string &v)`
Nustato vardą.
- `void setPavarde (const string &p)`
Nustato pavardę.
- virtual `void spausdinti (ostream &os) const =0`
Gryna virtuali funkcija — daro klasę abstrakčia.

Rule of Five

- `Zmogus (const Zmogus &)=default`
- `Zmogus (Zmogus &&) noexcept=default`
- `Zmogus & operator= (const Zmogus &)=default`
- `Zmogus & operator= (Zmogus &&) noexcept=default`
- virtual `~Zmogus ()=default`
Virtualus destruktorius.

Apsaugoti Atributai

- string `vardas_`
Žmogaus vardas.
- string `pavarde_`
Žmogaus pavardė

5.2.1 Smulkus aprašymas

Abstrakti bazinė klasė, aprašanti bendrus žmogaus atributus.

Klasė yra abstrakti dėl gryniosios virtualios funkcijos `spausdinti()`. Negalima sukurti `Zmogus` tipo objektų tiesiogiai — tik per paveldėjusias klases.

Pastaba

Destruktorius yra virtualus — būtina paveldėjimo hierarchijoje, kad ištrindami per bazinę rodyklę išvalytume ir išvestinio objekto laukus.

5.2.2 Konstruktoriaus ir Destruktoriaus Dokumentacija**5.2.2.1 Zmogus() [1/4]**

```
Zmogus::Zmogus () [default]
```

Numatytasis konstruktorius — sukuria tuščią objektą.

5.2.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    string v,
    string p) [inline]
```

Pilnas konstruktorius.

Parametrai

<i>v</i>	Vardas
<i>p</i>	Pavardė

5.2.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & ) [default]
```

5.2.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && ) [default], [noexcept]
```

5.2.2.5 ~Zmogus()

```
virtual Zmogus::~Zmogus () [virtual], [default]
```

Virtualus destruktorius.

Būtinās paveldėjimo hierarchijoje — be jo, ištrinus išvestinį objektą per Zmogus* rodyklę, jo papildomi laukai liktų neišvalyti.

5.2.3 Metodų Dokumentacija**5.2.3.1 operator=()** [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & ) [default]
```

5.2.3.2 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && ) [default], [noexcept]
```

5.2.3.3 pavarde()

```
const string & Zmogus::pavarde () const [inline]
```

Grąžina pavardę.

5.2.3.4 setPavarde()

```
void Zmogus::setPavarde (
    const string & p) [inline]
```

Nustato pavardę.

5.2.3.5 setVardas()

```
void Zmogus::setVardas (
    const string & v) [inline]
```

Nustato vardą.

5.2.3.6 spausdinti()

```
virtual void Zmogus::spausdinti (
    ostream & os) const [pure virtual]
```

Gryna virtuali funkcija — daro klasę abstrakčia.

Parametrai

<i>os</i>	Išvesties srautas, į kurį spausdinama.
-----------	--

Kiekviena paveldėjusi klasė privalo įgyvendinti šią funkciją.
Realizuota [Studentas](#).

5.2.3.7 vardas()

```
const string & Zmogus::vardas () const [inline]
```

Grąžina vardą.

5.2.4 Atributų Dokumentacija

5.2.4.1 pavarde_

```
string Zmogus::pavarde_ [protected]
```

Žmogaus pavardė

5.2.4.2 vardas_

```
string Zmogus::vardas_ [protected]
```

Žmogaus vardas.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [struktura.h](#)

skyrius 6

Failo Dokumentacija

6.1 funkcijos.cpp Failo Nuoroda

```
#include <windows.h>
#include "struktura.h"
```

Apibrėžimai

- #define `NOMINMAX`
- #define `WIN32_LEAN_AND_MEAN`

Funkcijos

- static std::mt19937 `mt` (std::chrono::steady_clock::now().time_since_epoch().count())
- string `genVarda` ()
Atsitiktinai parenka vardą iš fiksuoto sąrašo.
- string `genPavarde` (string vardas)
Pagal vardo galūnę parenka tinkamą pavardę (vyrišką ar moterišką).
- void `genPazymius` (vector< int > &paz, int &egz)
Sugeneruoja 20 atsitiktinių pažymių ir egzamino balą.
- void `genFaila` (const string &failas, int kiek)
Sukuria duomenų failą su nurodytu studentų kiekiu.
- int `gautiSkaiciu` (string info, int min, int max)
Saugiai skaito skaičių iš įvesties; bando iš naujo kol bus įvesta tinkama reikšmė.
- void `skaitytiVector` (const string &failas, vector< `Studentas` > &grupe, int metodas)
- void `skaitytiList` (const string &failas, list< `Studentas` > &grupe, int metodas)
- void `skaitytiDeque` (const string &failas, deque< `Studentas` > &grupe, int metodas)
- void `skirstytiVector` (const vector< `Studentas` > &visi, vector< `Studentas` > &kieti, vector< `Studentas` > &tinginiai)
- void `skirstytiList` (const list< `Studentas` > &visi, list< `Studentas` > &kieti, list< `Studentas` > &tinginiai)
- void `skirstytiDeque` (const deque< `Studentas` > &visi, deque< `Studentas` > &kieti, deque< `Studentas` > &tinginiai)

6.1.1 Apibrėžimų Dokumentacija

6.1.1.1 NOMINMAX

```
#define NOMINMAX
```

6.1.1.2 WIN32_LEAN_AND_MEAN

```
#define WIN32_LEAN_AND_MEAN
```

6.1.2 Funkcijos Dokumentacija

6.1.2.1 gautiSkaiciu()

```
int gautiSkaiciu (
    string info,
    int min,
    int max)
```

Saugiai skaito skaičių iš įvesties; bando iš naujo kol bus įvesta tinkama reikšmė.

6.1.2.2 genFaila()

```
void genFaila (
    const string & failas,
    int kiek)
```

Sukuria duomenų failą su nurodytu studentų kiekiu.

6.1.2.3 genPavarde()

```
string genPavarde (
    string vardas)
```

Pagal vardo galūnę parenka tinkamą pavardę (vyrišką ar moterišką).

6.1.2.4 genPazymius()

```
void genPazymius (
    vector< int > & paz,
    int & egz)
```

Sugeneruoja 20 atsitiktinių pažymių ir egzamino balą.

6.1.2.5 genVarda()

```
string genVarda ()
```

Atsitiktinai parenka vardą iš fiksuoto sąrašo.

6.1.2.6 mt()

```
std::mt19937 mt (
    std::chrono::steady_clock::now().time_since_epoch().count() ) [static]
```

6.1.2.7 skaitytiDeque()

```
void skaitytiDeque (
    const string & failas,
    deque< Studentas > & grupe,
    int metodas)
```

6.1.2.8 skaitytiList()

```
void skaitytiList (
    const string & failas,
    list< Studentas > & grupe,
    int metodas)
```

6.1.2.9 skaitytiVector()

```
void skaitytiVector (
    const string & failas,
    vector< Studentas > & grupe,
    int metodas)
```

6.1.2.10 skirstytiDeque()

```
void skirstytiDeque (
    const deque< Studentas > & visi,
    deque< Studentas > & kieti,
    deque< Studentas > & tinginiai)
```

6.1.2.11 skirstytiList()

```
void skirstytiList (
    const list< Studentas > & visi,
    list< Studentas > & kieti,
    list< Studentas > & tinginiai)
```

6.1.2.12 skirstytiVector()

```
void skirstytiVector (
    const vector< Studentas > & visi,
    vector< Studentas > & kieti,
    vector< Studentas > & tinginiai)
```

6.2 README.md Failo Nuoroda

6.3 struktura.h Failo Nuoroda

Klasių apibrėžimai: [Zmogus](#) (abstrakti bazė) ir [Studentas](#) (paveldėjusi).

```
#include <windows.h>
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <algorithm>
#include <random>
#include <chrono>
#include <limits>
#include <fstream>
#include <sstream>
#include <list>
#include <deque>
#include <filesystem>
```

Klasės

- class [Zmogus](#)
Abstrakti bazinė klasė, aprašanti bendrus žmogaus atributus.
- class [Studentas](#)
Paveldėjusi klasė iš [Zmogus](#) — atstovauja studentui su pažymiais ir egzaminu.

Apibrėžimai

- #define [NOMINMAX](#)
- #define [WIN32_LEAN_AND_MEAN](#)

Funkcijos

- ostream & [operator<<](#) (ostream &os, const [Zmogus](#) &z)

- Išvesties operatorius bet kuriam Zmogus tipui.*
- istream & operator>> (istream &is, Studentas &st)
 - Išvesties operatorius Studento objektui.*
- int gautiSkaiciu (string info, int min, int max)
 - Saugiai skaito skaičių iš įvesties; bando iš naujo kol bus įvesta tinkama reikšmė.*
- string genVarda ()
 - Atsitiktinai parenka vardą iš fiksuoto sąrašo.*
- string genPavarde (string vardas)
 - Pagal vardo galūnę parenka tinkamą pavardę (vyrišką ar moterišką).*
- void genPazymius (vector< int > &paz, int &egz)
 - Sugeneruoja 20 atsitiktinių pažymių ir egzamino balą.*
- void genFaila (const string &failas, int kiek)
 - Sukuria duomenų failą su nurodytu studentų kiekiu.*
- void skaitytiVector (const string &failas, vector< Studentas > &grupe, int metodas)
- void skaitytiList (const string &failas, list< Studentas > &grupe, int metodas)
- void skaitytiDeque (const string &failas, deque< Studentas > &grupe, int metodas)
- void skirstytiVector (const vector< Studentas > &visi, vector< Studentas > &kieti, vector< Studentas > &tinginiai)
- void skirstytiList (const list< Studentas > &visi, list< Studentas > &kieti, list< Studentas > &tinginiai)
- void skirstytiDeque (const deque< Studentas > &visi, deque< Studentas > &kieti, deque< Studentas > &tinginiai)
- void skaitytiFaila (const string &failas, vector< Studentas > &grupe, int metodas)
- void spausdintiRezultatus (const vector< Studentas > &grupe, int rodyti, const string &failas)
- void splitStudents (const vector< Studentas > &visi, vector< Studentas > &kieti, vector< Studentas > &tinginiai, int metodas)
- void vykdytiVector ()
- void test1 ()
- void test2 (const string &failas, int metodas)

6.3.1 Smulkus aprašymas

Klasių apibrėžimai: Zmogus (abstrakti bazė) ir Studentas (paveldėjusi).

Versija

2.0

Šiame faile aprašyta visa klasių hierarchija ir laisvų funkcijų prototipai.

6.3.2 Apibrėžimų Dokumentacija

6.3.2.1 NOMINMAX

```
#define NOMINMAX
```

6.3.2.2 WIN32_LEAN_AND_MEAN

```
#define WIN32_LEAN_AND_MEAN
```

6.3.3 Funkcijos Dokumentacija

6.3.3.1 gautiSkaiciu()

```
int gautiSkaiciu (
    string info,
    int min,
    int max)
```

Saugiai skaito skaičių iš įvesties; bando iš naujo kol bus įvesta tinkama reikšmė.

6.3.3.2 genFaila()

```
void genFaila (
    const string & failas,
    int kiek)
```

Sukuria duomenų failą su nurodytu studentų kiekiu.

6.3.3.3 genPavarde()

```
string genPavarde (
    string vardas)
```

Pagal vardo galūnę parenka tinkamą pavardę (vyrišką ar moterišką).

6.3.3.4 genPazymius()

```
void genPazymius (
    vector< int > & paz,
    int & egz)
```

Sugeneruoja 20 atsitiktinių pažymių ir egzamino balą.

6.3.3.5 genVarda()

```
string genVarda ()
```

Atsitiktinai parenka vardą iš fiksuoto sąrašo.

6.3.3.6 operator<<()

```
ostream & operator<< (
    ostream & os,
    const Zmogus & z) [inline]
```

Išvesties operatorius bet kuriam [Zmogus](#) tipui.

Parametrai

<i>os</i>	Išvesties srautas
<i>z</i>	Žmogaus nuoroda (gali būti Studentas)

Gražina

Nuoroda į srautą

Per virtualų dispatch automatiškai parinks teisingą spausdinti() versiją.

6.3.3.7 operator>>()

```
istream & operator>> (
    istream & is,
    Studentas & st) [inline]
```

Išvesties operatorius Studento objektui.

Parametrai

<i>is</i>	Išvesties srautas
<i>st</i>	Studentas , į kurį skaitoma

Gražina

Nuoroda į srautą

Tikisi vienos eilutės: Vardas Pavardė ND1 ND2 ... NDn Egzaminas Paskutinis skaičius laikomas egzamino balu.

6.3.3.8 skaitytiDeque()

```
void skaitytiDeque (
    const string & failas,
    deque< Studentas > & grupe,
    int metodas)
```

6.3.3.9 skaitytiIsFailo()

```
void skaitytiIsFailo (
    const string & failas,
    vector< Studentas > & grupe,
    int metodas)
```

6.3.3.10 skaitytiList()

```
void skaitytiList (
    const string & failas,
    list< Studentas > & grupe,
    int metodas)
```

6.3.3.11 skaitytiVector()

```
void skaitytiVector (
    const string & failas,
    vector< Studentas > & grupe,
    int metodas)
```

6.3.3.12 skirstytiDeque()

```
void skirstytiDeque (
    const deque< Studentas > & visi,
    deque< Studentas > & kieti,
    deque< Studentas > & tinginiai)
```

6.3.3.13 skirstytiList()

```
void skirstytiList (
    const list< Studentas > & visi,
    list< Studentas > & kieti,
    list< Studentas > & tinginiai)
```

6.3.3.14 skirstytiVector()

```
void skirstytiVector (
    const vector< Studentas > & visi,
    vector< Studentas > & kieti,
    vector< Studentas > & tinginiai)
```

6.3.3.15 spausdintiRezultatus()

```
void spausdintiRezultatus (
    const vector< Studentas > & grupe,
    int rodyti,
    const string & failas)
```


6.3.3.16 splitStudents()

```
void splitStudents (
    const vector< Studentas > & visi,
    vector< Studentas > & kieti,
    vector< Studentas > & tinginiai,
    int metodas)
```

6.3.3.17 test1()

```
void test1 ()
```

6.3.3.18 test2()

```
void test2 (
    const string & failas,
    int metodas)
```

6.3.3.19 vykdytiVector()

```
void vykdytiVector ()
```

6.4 struktura.h

[Eiti į šio failo dokumentaciją.](#)

```
00001
00008
00009 #pragma once
00010
00011 #define NOMINMAX
00012 #define WIN32_LEAN_AND_MEAN
00013 #include <windows.h>
00014
00015 #include <iostream>
00016 #include <vector>
00017 #include <string>
00018 #include <iomanip>
00019 #include <algorithm>
00020 #include <random>
00021 #include <chrono>
00022 #include <limits>
00023 #include <fstream>
00024 #include <sstream>
00025 #include <list>
00026 #include <deque>
00027 #include <filesystem>
00028
00029 using std::cin;
00030 using std::cout;
00031 using std::string;
00032 using std::vector;
00033 using std::left;
00034 using std::right;
00035 using std::setw;
00036 using std::endl;
00037 using std::sort;
00038 using std::fixed;
00039 using std::setprecision;
00040 using std::ifstream;
00041 using std::ofstream;
00042 using std::getline;
00043 using std::cerr;
00044 using std::stringstream;
00045 using std::move;
00046 using std::ostream;
00047 using std::istream;
00048 using std::chrono::high_resolution_clock;
00049 using std::chrono::duration_cast;
00050 using std::chrono::duration;
00051 using std::chrono::milliseconds;
00052 using std::numeric_limits;
00053 using std::mt19937;
00054 using std::chrono::steady_clock;
00055 using std::invalid_argument;
00056 using std::out_of_range;
```

```

00057 using std::exception;
00058 using std::runtime_error;
00059 using std::streamsize;
00060 using std::to_string;
00061 using std::pair;
00062 using std::list;
00063 using std::deque;
00064 using std::stable_partition;
00065 using std::copy_if;
00066 using std::back_inserter;
00067 using std::make_move_iterator;
00068 namespace fs = std::filesystem;
00069
00080 class Zmogus {
00081 protected:
00082     string vardas_;
00083     string pavarde_;
00084
00085 public:
00086     Zmogus() = default;
00087
00088     Zmogus(string v, string p)
00089         : vardas_(move(v)), pavarde_(move(p)) {}
00090
00091     Zmogus(const Zmogus&) = default;
00092     Zmogus(Zmogus&&) noexcept = default;
00093     Zmogus& operator=(const Zmogus&) = default;
00094     Zmogus& operator=(Zmogus&&) noexcept = default;
00095     virtual ~Zmogus() = default;
00096
00097     const string& vardas() const { return vardas_; }
00098     const string& pavarde() const { return pavarde_; }
00099
00100     void setVardas(const string& v) { vardas_ = v; }
00101     void setPavarde(const string& p) { pavarde_ = p; }
00102
00103     virtual void spausdinti(ostream& os) const = 0;
00104 };
00105
00106 class Studentas : public Zmogus {
00107 private:
00108     vector<int> paz_;
00109     int egz_ = 0;
00110     double gal_vid_ = 0.0;
00111     double gal_med_ = 0.0;
00112
00113 public:
00114     Studentas() = default;
00115
00116     Studentas(string v, string p, vector<int> paz, int egz, int metodas)
00117         : Zmogus(move(v), move(p)),
00118           paz_(move(paz)), egz_(egz)
00119     {
00120         apskaiciuoti(metodas);
00121     }
00122
00123     ~Studentas() override = default;
00124
00125     Studentas(const Studentas& kitas)
00126         : Zmogus(kitas),
00127           paz_(kitas.paz_), egz_(kitas.egz_),
00128           gal_vid_(kitas.gal_vid_), gal_med_(kitas.gal_med_)
00129     {}
00130
00131     Studentas& operator=(const Studentas& kitas) {
00132         if (this != &kitas) {
00133             Zmogus::operator=(kitas);
00134             paz_ = kitas.paz_;
00135             egz_ = kitas.egz_;
00136             gal_vid_ = kitas.gal_vid_;
00137             gal_med_ = kitas.gal_med_;
00138         }
00139         return *this;
00140     }
00141
00142     Studentas(Studentas&& kitas) noexcept
00143         : Zmogus(move(kitas)),
00144           paz_(move(kitas.paz_)),
00145           egz_(kitas.egz_), gal_vid_(kitas.gal_vid_), gal_med_(kitas.gal_med_)
00146     {
00147         kitas.egz_ = 0;
00148         kitas.gal_vid_ = 0.0;
00149         kitas.gal_med_ = 0.0;
00150     }
00151
00152     Studentas& operator=(Studentas&& kitas) noexcept {

```

```

00219         if (this != &kitas) {
00220             Zmogus::operator=(move(kitas));
00221             paz_ = move(kitas.paz_);
00222             egz_ = kitas.egz_;
00223             gal_vid_ = kitas.gal_vid_;
00224             gal_med_ = kitas.gal_med_;
00225             kitas.egz_ = 0;
00226             kitas.gal_vid_ = 0.0;
00227             kitas.gal_med_ = 0.0;
00228         }
00229         return *this;
00230     }
00231
00232 void spausdinti(ostream& os) const override {
00240     os << left << setw(15) << vardas_
00241         << setw(15) << pavarde_;
00242     os << "ND:";
00243     for (int p : paz_) os << " " << p;
00244     os << " Egz: " << egz_;
00245     if (gal_vid_ > 0.0)
00246         os << fixed << setprecision(2) << " Vid: " << gal_vid_;
00247     if (gal_med_ > 0.0)
00248         os << fixed << setprecision(2) << " Med: " << gal_med_;
00249 }
00250
00251 const vector<int>& paz() const { return paz_; }
00252 int egz() const { return egz_; }
00253 double galVid() const { return gal_vid_; }
00254 double galMed() const { return gal_med_; }
00255
00256 void setEgz(int e) { egz_ = e; }
00257 void addPazmys(int p) { paz_.push_back(p); }
00258
00259 void nustatytiEgzIsGalo() {
00260     if (!paz_.empty()) {
00261         egz_ = paz_.back();
00262         paz_.pop_back();
00263     }
00264 }
00265
00266 double skaiciuotiVidurki() const {
00267     if (paz_.empty()) return 0.0;
00268     double suma = 0.0;
00269     for (int p : paz_) suma += p;
00270     return suma / static_cast<double>(paz_.size());
00271 }
00272
00273 double skaiciuotiMediana() const {
00274     if (paz_.empty()) return 0.0;
00275     vector<int> tmp = paz_;
00276     sort(tmp.begin(), tmp.end());
00277     size_t n = tmp.size();
00278     if (n % 2 == 0) return (tmp[n / 2 - 1] + tmp[n / 2]) / 2.0;
00279     return tmp[n / 2];
00280 }
00281
00282 void apskaiciuoti(int metodos) {
00283     if (metodos == 1 || metodos == 3)
00284         gal_vid_ = skaiciuotiVidurki() * 0.4 + egz_ * 0.6;
00285     if (metodos == 2 || metodos == 3)
00286         gal_med_ = skaiciuotiMediana() * 0.4 + egz_ * 0.6;
00287 }
00288
00289 friend istream& operator>>(istream& is, Studentas& st);
00290 };
00291
00292 inline ostream& operator<<(ostream& os, const Zmogus& z) {
00293     z.spausdinti(os);
00294     return os;
00295 }
00296
00297 inline istream& operator>>(istream& is, Studentas& st) {
00298     string eilute;
00299     if (!getline(is, eilute)) return is;
00300     if (eilute.empty()) return is;
00301
00302     stringstream ss(eilute);
00303     string v, p;
00304     if (!(ss >> v >> p)) return is;
00305
00306     st.vardas_ = v;
00307     st.pavarde_ = p;
00308     st.paz_.clear();
00309     st.egz_ = 0;
00310     st.gal_vid_ = 0.0;
00311     st.gal_med_ = 0.0;
00312 }

```

```

00357     int n;
00358     while (ss » n) st.paz_.push_back(n);
00359     if (!st.paz_.empty()) {
00360         st.egz_ = st.paz_.back();
00361         st.paz_.pop_back();
00362     }
00363     return is;
00364 }
00365
00366 // =====
00367 // Laisvos funkcijos
00368 // =====
00369
00371 int     gautiSkaiciu(string info, int min, int max);
00372
00374 string genVarda();
00375
00377 string genPavarde(string vardas);
00378
00380 void    genPazymius(vector<int>& paz, int& egz);
00381
00383 void    genFaila(const string& failas, int kiek);
00384
00385 void    skaitytiVector(const string& failas, vector<Studentas>& grupe, int metodos);
00386 void    skaitytiList(const string& failas, list <Studentas>& grupe, int metodos);
00387 void    skaitytiDeque(const string& failas, deque <Studentas>& grupe, int metodos);
00388
00389 void    skirstytiVector(const vector<Studentas>& visi, vector<Studentas>& kieti, vector<Studentas>&
tinginiai);
00390 void    skirstytiList(const list <Studentas>& visi, list <Studentas>& kieti, list <Studentas>&
tinginiai);
00391 void    skirstytiDeque(const deque <Studentas>& visi, deque <Studentas>& kieti, deque <Studentas>&
tinginiai);
00392
00393 void    skaitytiIsFailo(const string& failas, vector<Studentas>& grupe, int metodos);
00394 void    spausdintiRezultatus(const vector<Studentas>& grupe, int rodyti, const string& failas);
00395 void    splitStudents(const vector<Studentas>& visi, vector<Studentas>& kieti,
vector<Studentas>& tinginiai, int metodos);
00396 void    vykdytiVector();
00397
00399 void    test1();
00400 void    test2(const string& failas, int metodos);

```

6.5 Uzd2.cpp Failo Nuoroda

```
#include "struktura.h"
```

Funkcijos

- int [main](#) ()

6.5.1 Funkcijos Dokumentacija

6.5.1.1 main()

```
int main ()
```

6.6 vektorius.cpp Failo Nuoroda

```
#include "struktura.h"
```

Funkcijos

- void [skaitytiIsFailo](#) (const string &failas, vector< [Studentas](#) > &grupe, int metodos)
- void [spausdintiRezultatus](#) (const vector< [Studentas](#) > &grupe, int rodyti, const string &failas)
- void [splitStudents](#) (const vector< [Studentas](#) > &visi, vector< [Studentas](#) > &kieti, vector< [Studentas](#) > &tinginiai, int metodos)
- void [vykdytiVector](#) ()

Kintamieji

- static const string `DATA_DIR` = "Data/"

6.6.1 Funkcijos Dokumentacija**6.6.1.1 skaitytiIsFailo()**

```
void skaitytiIsFailo (  
    const string & failas,  
    vector< Studentas > & grupe,  
    int metodas)
```

6.6.1.2 spausdintiRezultatus()

```
void spausdintiRezultatus (  
    const vector< Studentas > & grupe,  
    int rodyti,  
    const string & failas)
```

6.6.1.3 splitStudents()

```
void splitStudents (  
    const vector< Studentas > & visi,  
    vector< Studentas > & kieti,  
    vector< Studentas > & tinginiai,  
    int metodas)
```

6.6.1.4 vykdytiVector()

```
void vykdytiVector ()
```

6.6.2 Kintamojo Dokumentacija**6.6.2.1 DATA_DIR**

```
const string DATA_DIR = "Data/" [static]
```

